Milestone 8: Add Multimodal Capabilities – Gradient Descenters

Current State of AdventureWorks Database Agent

Prior to the implementation of milestone 8, our database agent implemented the following features

- Natural language Understanding
    - Accepts business questions in natural language
- Function calling, incorporating tools into the prompts which
    - Retrieved database schema, execute SQL queries and returned data frames, checked for T-SQL syntax and validity
    - These were hardcoded python functions whose results were incorporated into the text prompts fed to the LLM, not function calling where the LLM itself would decide which to use. In future state, we will modify these to actual function calls
- RAG Enhanced Context Retrieval
    - Contextual information specific to the AdventureWorks2025DW database, including schema's, business rules, join rules, patterns, and example queries
- Evaluation Framework to Ensure Accuracy & Privacy

With the addition of milestone 8, we added the following multimodality features, as well as built out a front end for using our AdventureWorks Database Agent. These features include

- Visualization generator
    - Supports 5 types of visualizations (line, bar, horizontal bar, pie, and heatmap) according to the type of user-query. This is generated in addition to the SQL query
- Vision Analyzer
    - Integrates OpenAI GPT-4o vision model to interpret the generated charts/graphs with specific prompts for each chart type to get contextual analysis
    - Extracts trends, patterns, and business insights from visualizations
- Redesigned agent framework
    - Redesigned framework orchestrates the multimodal pipeline, integrating the existing RAG-enabled SQL generation with the new visualization and vision analysis capabilities
- Streamlit Front End

    - Web application where users will type in their queries, and get back the SQL response, associated visuals, chart analysis & insights, as well as performance metrics & query history

Cross-Modal Consistency Testing

Validated that vision analysis aligns with actual data values by comparing AI insights against ground-truth data. Tested with the following suite and received an overall 89% level of accuracy in our moderate queries.

- Value Accuracy, Ranking Consistency, Trend Direction, Count Accuracy

For validation, you can examine the example shown in appendix A, where the query "Show me the monthly sales trends for 2013" generated a SQL statement, dataframe, and visualization, along with AI insights from analyzing the visual. The AI insights line up with a very high degree of accuracy with what is shown in the dataframe and the line-chart.

Modality Specific Prompting Strategies

For visual analysis, we implemented specific prompts for each type of chart. This is so that the user can receive tailored analysis optimized for a particular visualization.

- Line Charts
    - Focus: Trend analysis, pattern recognition, growth metrics, key inflection points
    - Output: Trends (direction), Patterns (seasonality), Key Points (peaks/troughs), Insights (business implications)
- Bar Charts (Vertical & Horizontal)
    - Focus: Ranking analysis, comparative insights, distribution patterns, outlier detection, market share estimation
    - Output: Leaders (top performers), Laggards (underperformers), Distribution (concentrated vs. spread), Insights (recommendations)
- Pie Charts
    - Focus: Market share percentages, segment dominance, balance assessment, growth opportunities
    - Output: Dominant Segments, Distribution balance, Small segments, Strategic insights
- Scatter Plots
    - Focus: Correlation strength/direction, cluster identification, outlier detection
    - Output: Correlation description, Pattern clusters, Outlier points, Business implications

Appendix:

Streamlit Demonstration of Agent + MultiModal Capabilities

AdventureWorks Multimodal Database Agent

Natural Language → SQL → Visualization → AI Insights

Ask a question about the sales data:

Show monthly sales trends for 2013

Process Query

Advanced Options

Generate Visualization

Enable Vision Analysis

Force Chart Type (optional)

Auto

Show Generated SQL

Generated SQL

```sql
SELECT
    d.CalendarYear,
    d.MonthNumberOfYear,
    d.EnglishMonthName,
    SUM(fis.SalesAmount) AS MonthlyRevenue,
    COUNT(DISTINCT fis.SalesOrderNumber) AS OrderCount
FROM dbo.FactInternetSales fis
INNER JOIN dbo.DimDate d ON fis.OrderDateKey = d.DateKey
WHERE d.CalendarYear = 2013
GROUP BY d.CalendarYear, d.MonthNumberOfYear, d.EnglishMonthName
ORDER BY d.MonthNumberOfYear
```

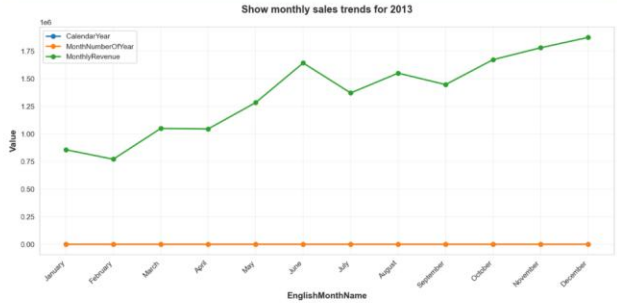
Query Results

Returned 12 rows in 15.56s

	CalendarYear	MonthNumberOfYear	EnglishMonthName	MonthlyRevenue	OrderCount
0	2013	1	January	857689.91	632
1	2013	2	February	771348.74	1421
2	2013	3	March	1049907.39	1690
3	2013	4	April	1046022.77	1612
4	2013	5	May	1284592.93	1792
5	2013	6	June	1643177.78	2007
6	2013	7	July	1371675.81	1875
7	2013	8	August	1551065.56	1966
8	2013	9	September	1447495.69	1894
9	2013	10	October	1673293.41	2131

Chart Type: line

The `use_column_width` parameter has been deprecated and will be removed in a future release. Please utilize the `use_container_width` parameter instead.



AI Visual Insights

AI Visual Insights

Trends: The overall trend for monthly sales revenue in 2013 shows an upward trajectory. There is a noticeable increase in revenue from the beginning to the end of the year.
Patterns: There appears to be a seasonal pattern, with sales dipping in the early months and then gradually increasing. This suggests a potential seasonal influence on sales, possibly linked to cycling seasonality or consumer buying behavior.

Key Points:

- **January:** Revenue is at its lowest point.
- **June:** A significant peak in sales, indicating a mid-year boost.
- **December:** The highest revenue, suggesting strong end-of-year sales.

Insights:

- **Marketing Strategy:** Focus marketing efforts in the months leading up to June and December to capitalize on peak sales periods.
- **Inventory Management:** Ensure adequate stock levels before June and December to meet increased demand.
- **Promotional Campaigns:** Consider launching promotions or discounts in the early months to boost sales during traditionally slower periods.
- **Product Launches:** Plan new product launches around May and November to align with rising sales trends.

These insights can guide strategic decisions to optimize sales and improve overall business performance.

Vision tokens used: 2011

Performance Metrics

SQL Generation	Total Time	Total Tokens
3.46s	15.56s	3847

Query History

Show monthly sales trends for 2013

Status: Success

Rows: 12

Time: 15.56s

AI Insights: Generated

Visualization Generation Functions:

```python
def detect_chart_type(self, df: pd.DataFrame, query_context: str = "") -> str:
    """
    Automatically detect the most appropriate chart type based on data characteristics

    Returns: 'line', 'bar', 'horizontal_bar', 'pie', 'scatter', 'heatmap', or 'table'
    """
    if df.empty or len(df) == 0:
        return 'table'

    num_rows = len(df)
    num_cols = len(df.columns)

    # Check column types
    numeric_cols = df.select_dtypes(include=['int64', 'float64', 'int32', 'float32']).columns
    categorical_cols = df.select_dtypes(include=['object', 'category']).columns
    datetime_cols = df.select_dtypes(include=['datetime64']).columns

    # Detect time series data
    time_indicators = ['year', 'month', 'day', 'date', 'quarter', 'week']
    has_time_column = any(
        any(indicator in col.lower() for indicator in time_indicators)
        for col in df.columns
    )

    # Query context hints
    context_lower = query_context.lower()

    # Decision tree for chart selection
    if has_time_column or len(datetime_cols) > 0 or 'trend' in context_lower or 'over time' in context_lower:
        return 'line'

    elif 'compare' in context_lower or 'versus' in context_lower or 'vs' in context_lower:
        if num_rows > 10:
            return 'horizontal_bar'
        return 'bar'

    elif ('top' in context_lower or 'bottom' in context_lower) and num_rows <= 20:
        if num_rows > 10:
            return 'horizontal_bar'
        return 'bar'

    elif 'distribution' in context_lower or 'breakdown' in context_lower:
        if num_rows <= 8 and len(numeric_cols) > 0:
            return 'pie'
        return 'bar'

    elif 'category' in context_lower or 'by category' in context_lower:
        return 'bar'

    elif len(numeric_cols) >= 2 and num_rows <= 100:
        return 'scatter'

    # Default heuristics based on data shape
    elif num_rows <= 8 and len(numeric_cols) == 1:
        return 'pie'

    elif num_rows <= 30:
        return 'bar'

    elif num_rows > 30:
        return 'horizontal_bar'

    else:
        return 'table'
```

```python
def create_visualization(
    self,
    df: pd.DataFrame,
    query_context: str = "",
    chart_type: Optional[str] = None,
    title: Optional[str] = None
) -> Tuple[str, str]:
    """
    Create visualization and save to file

    Args:
        df: DataFrame with query results
        query_context: The original natural language query
        chart_type: Force specific chart type (optional)
        title: Custom title (optional)

    Returns:
        (file_path, chart_type_used)
    """
    if df.empty:
        print("⚠  No data to visualize")
        return None, 'empty'

    # Auto-detect chart type if not specified
    if chart_type is None:
        chart_type = self.detect_chart_type(df, query_context)

    # Generate title if not provided
    if title is None:
        title = query_context if query_context else "Query Results"

    # Create figure
    fig, ax = plt.subplots(figsize=(12, 6))

    try:
        if chart_type == 'line':
            self._create_line_chart(df, ax, title)

        elif chart_type == 'bar':
            self._create_bar_chart(df, ax, title)

        elif chart_type == 'horizontal_bar':
            self._create_horizontal_bar_chart(df, ax, title)

        elif chart_type == 'pie':
            self._create_pie_chart(df, ax, title)

        elif chart_type == 'scatter':
            self._create_scatter_plot(df, ax, title)

        elif chart_type == 'heatmap':
            self._create_heatmap(df, ax, title)

        else:
            # No chart, return None
            plt.close(fig)
            return None, 'table'

        # Save figure
        timestamp = pd.Timestamp.now().strftime("%Y%m%d_%H%M%S")
        filename = f"chart_{chart_type}_{timestamp}.png"
        filepath = self.output_dir / filename

        plt.tight_layout()
        plt.savefig(filepath, dpi=150, bbox_inches='tight', facecolor='white')
        plt.close(fig)

        print(f"✓ Visualization saved: {filepath}")
        return str(filepath), chart_type

    except Exception as e:
        print(f"✗ Error creating visualization: {e}")
        plt.close(fig)
        return None, 'error'
```

- 6 functions for each type of chart (line, bar, horizontal bar, pie, scatter, heatmap) – Not shown here

AI Vision Analysis

Encode Image to Base65 for API Transmission

```python
def encode_image(self, image_path: str) -> str:
    """Encode image to base64 for API transmission"""
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode('utf-8')
```

Create Analysis Prompts for Chart-Type Specific Prompts

```python
    def create_analysis_prompt(
        self,
        chart_type: str,
        query_context: str,
        data_summary: Dict
    ) -> str:
        """
        Create tailored prompt for vision model based on chart type

        This implements modality-specific prompt strategies (Requirement 4)
        """
        base_prompt = f"""You are analyzing a data visualization from a sales analytics dashboard for AdventureWorks company (bike/cycling products).

**Context:**
- Original Query: "{query_context}"
- Chart Type: {chart_type}
- Data Points: {data_summary.get('rows', 'Unknown')} rows
- Metrics: {', '.join(data_summary.get('column_names', []))}

**Your Task:**
Analyze this visualization and provide business insights. Focus on:

"""

        # Chart-type-specific analysis instructions
        if chart_type == 'line':
            prompt = base_prompt + """1. **Trend Analysis:** Identify overall trends (upward, downward, flat, seasonal)
2. **Pattern Recognition:** Spot recurring patterns, cycles, or anomalies
3. **Key Moments:** Highlight peaks, troughs, or inflection points
4. **Growth Metrics:** Estimate growth rates or changes between periods
5. **Business Insights:** What do these trends mean for business decisions?

**Format:**
- Trends: [Describe overall direction]
- Patterns: [Note any repeating patterns or seasonality]
- Key Points: [Identify significant data points]
- Insights: [Provide actionable business insights]
"""

        elif chart_type in ['bar', 'horizontal_bar']:
            prompt = base_prompt + """1. **Ranking Analysis:** Identify top and bottom performers
2. **Comparative Insights:** Compare relative performance across categories
3. **Distribution:** Analyze how values are distributed (concentrated vs. spread)
4. **Outliers:** Spot any unusual performers (exceptionally high or low)
5. **Market Share:** Estimate contribution of top performers to total
6. **Business Insights:** Recommendations based on performance gaps

**Format:**
- Leaders: [Identify top performers]
- Laggards: [Identify underperformers]
- Distribution: [Describe how values are spread]
- Insights: [Actionable business recommendations]
"""

        elif chart_type == 'pie':
            prompt = base_prompt + """1. **Market Share:** Calculate percentage contributions of each segment
2. **Dominance:** Identify which segments dominate and which are marginal
3. **Balance:** Assess whether distribution is balanced or concentrated
4. **Opportunities:** Spot segments with growth potential
5. **Business Insights:** Strategic recommendations based on distribution

**Format:**
- Dominant Segments: [Which segments lead?]
- Distribution: [Balanced or concentrated?]
- Small Segments: [Minor contributors worth noting]
- Insights: [Strategic recommendations]
"""

        elif chart_type == 'scatter':
            prompt = base_prompt + """1. **Correlation:** Identify relationship between variables (positive, negative, none)
2. **Clusters:** Spot groupings or clusters in the data
3. **Outliers:** Identify data points that don't fit the pattern
4. **Strength:** Assess how strong the relationship is
5. **Business Insights:** What does this relationship mean for operations?

**Format:**
- Correlation: [Describe relationship]
- Patterns: [Note clusters or groupings]
- Outliers: [Identify unusual points]
- Insights: [Business implications]
"""

        else:
            # Generic analysis for other chart types
            prompt = base_prompt + """1. **Key Observations:** What stands out visually in this chart?
2. **Data Patterns:** Identify any clear patterns or structures
3. **Notable Values:** Highlight extreme or interesting values
4. **Comparisons:** Compare different elements shown
5. **Business Insights:** What actions should be taken based on this data?

**Format:**
- Observations: [Key visual findings]
- Patterns: [Any clear patterns]
- Insights: [Business recommendations]
"""

        # Add data context if available
        if 'numeric_summary' in data_summary:
            prompt += f"\n**Data Context:**\n"
            for col, stats in data_summary['numeric_summary'].items():
                if stats.get('total'):
                    prompt += f"- {col}: Min={stats['min']:,.0f}, Max={stats['max']:,.0f}, Avg={stats['mean']:,.0f}, Total={stats['total']:,.0f}\n"

        prompt += "\n**Important:** Be specific with values you see in the chart. Provide actionable insights, not just descriptions."

        return prompt
```

Analyze Visualizations

```python
def analyze_visualization(
    self,
    image_path: str,
    chart_type: str,
    query_context: str,
    data_summary: Dict,
    detail_level: str = "high"
) -> Dict:
    """
    Analyze visualization using vision model

    Args:
        image_path: Path to chart image
        chart_type: Type of chart (line, bar, pie, etc.)
        query_context: Original natural language query
        data_summary: Summary statistics about the data
        detail_level: Vision detail level ("low", "high", "auto")

    Returns:
        Dictionary with analysis results and metadata
    """
    if not Path(image_path).exists():
        return {
            'success': False,
            'error': f"Image file not found: {image_path}"
        }

    try:
        # Encode image
        base64_image = self.encode_image(image_path)

        # Create tailored prompt
        prompt = self.create_analysis_prompt(chart_type, query_context, data_summary)

        # Call vision model
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "text",
                            "text": prompt
                        },
                        {
                            "type": "image_url",
                            "image_url": {
                                "url": f"{base64_image}",
                                "detail": detail_level
                            }
                        }
                    ]
                }
            ],
            max_tokens=800,
            temperature=0.3  # Low temperature for more focused analysis
        )

        # Extract analysis
        analysis_text = response.choices[0].message.content

        # Store analysis result
        result = {
            'success': True,
            'analysis': analysis_text,
            'chart_type': chart_type,
            'query_context': query_context,
            'model': self.model,
            'tokens': {
                'prompt': response.usage.prompt_tokens,
                'completion': response.usage.completion_tokens,
                'total': response.usage.total_tokens
            }
        }

        # Save to history
        self.analysis_history.append(result)

        return result

    except Exception as e:
        return {
            'success': False,
            'error': str(e),
            'chart_type': chart_type
        }
```

Compare Visualizations

```python
    def compare_visualizations(
        self,
        image_paths: List[str],
        comparison_context: str,
        chart_descriptions: List[str]
    ) -> Dict:
        """
        Compare multiple visualizations to find cross-cutting insights

        This demonstrates advanced multimodal analysis
        """
        if len(image_paths) < 2:
            return {'success': False, 'error': 'Need at least 2 images to compare'}

        try:
            # Encode all images
            encoded_images = [self.encode_image(path) for path in image_paths]

            # Create comparison prompt
            prompt = f"""You are comparing multiple data visualizations from AdventureWorks sales analytics.

**Context:** {comparison_context}

**Charts:**
"""
            for i, desc in enumerate(chart_descriptions, 1):
                prompt += f"{i}. {desc}\n"

            prompt += """
**Your Task:**
Compare these visualizations and provide:
1. **Common Patterns:** Insights that appear across multiple charts
2. **Contrasts:** Differences or contradictions between charts
3. **Cross-Analysis:** Insights that emerge only when viewing charts together
4. **Integrated Insights:** Overall conclusions from all visualizations
5. **Recommendations:** Strategic actions based on the complete picture

Be specific and reference which charts show which patterns.
"""

            # Build content with all images
            content = [{"type": "text", "text": prompt}]
            for encoded_image in encoded_images:
                content.append({
                    "type": "image_url",
                    "image_url": {
                        "url": f"{encoded_image}",
                        "detail": "high"
                    }
                })

            # Call vision model
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[{"role": "user", "content": content}],
                max_tokens=1000,
                temperature=0.3
            )

            return {
                'success': True,
                'comparative_analysis': response.choices[0].message.content,
                'num_charts': len(image_paths),
                'tokens': {
                    'prompt': response.usage.prompt_tokens,
                    'completion': response.usage.completion_tokens,
                    'total': response.usage.total_tokens
                }
            }

        except Exception as e:
            return {'success': False, 'error': str(e)}
```

Performance Metrics

```python
def get_analysis_summary(self) -> Dict:
    """Get summary of all analyses performed"""
    if not self.analysis_history:
        return {'total_analyses': 0}

    total_tokens = sum(a['tokens']['total'] for a in self.analysis_history if a['success'])

    return {
        'total_analyses': len(self.analysis_history),
        'successful': sum(1 for a in self.analysis_history if a['success']),
        'failed': sum(1 for a in self.analysis_history if not a['success']),
        'total_tokens_used': total_tokens,
        'chart_types_analyzed': list(set(a['chart_type'] for a in self.analysis_history if a['success']))
    }
```